

MATERIAL DE APOIO

POSICIONAMENTO

O posicionamento padrão dos elementos na página segue a declaração feita no HTML. A propriedade **position** do CSS é usada para permitir que a posição dos elementos seja alterada. Os possíveis valores são: **static**, **relative**, **absolute**, **fixed** e **sticky**. O valor padrão é o **static**, que respeita a posição pelo HTML.

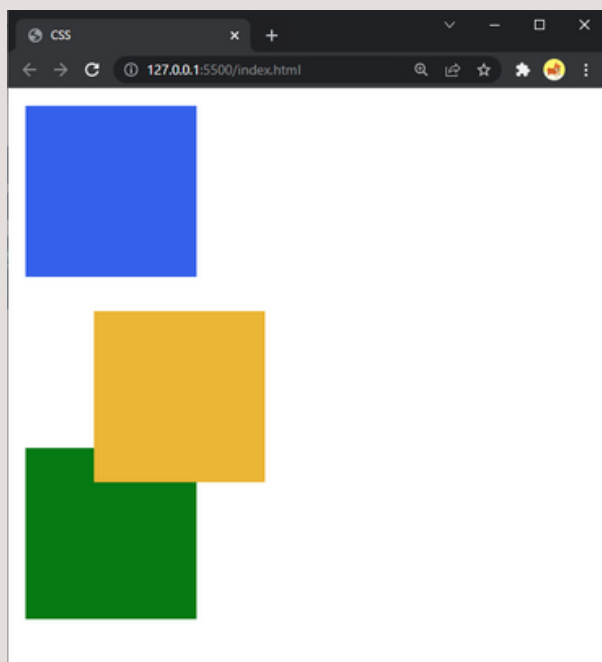
```
#square1 {  
  position: relative;  
  position: absolute;  
  position: fixed;  
  position: sticky;  
}
```

RELATIVE

A posição **relative** se baseia inicialmente na posição **static**, com a possibilidade de utilização das seguintes propriedades: **top**, **bottom**, **left** e **right**. Essas propriedades permitem alterar o deslocamento inicial do componente, indicando a distância a partir do topo, do final do bloco, do lado esquerdo e do direito, respectivamente.

Caso nenhuma dessas propriedades seja informada em um elemento com `position: relative`, o comportamento será o mesmo do `static`.

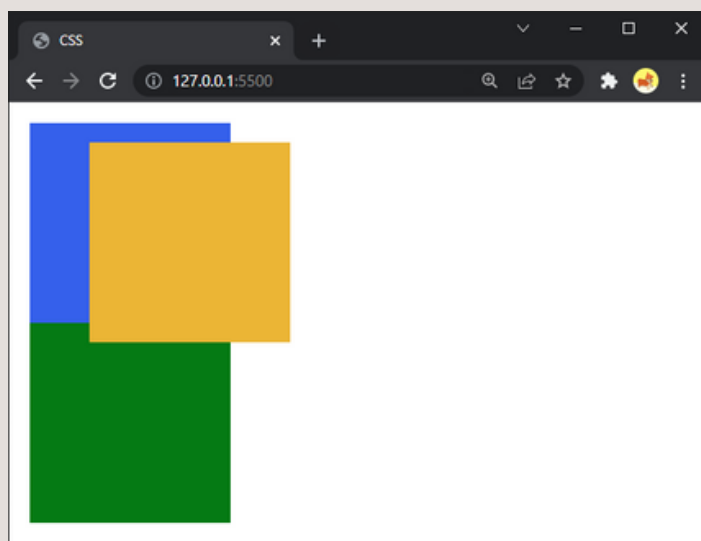
```
#square2 {  
  position: relative;  
  top: 20px;  
  left: 40px;  
}
```



ABSOLUTE

A posição **absolute** permite que o elemento seja posicionado livremente na tela, baseado nas propriedades **top**, **bottom**, **left** e **right**. O posicionamento inicial do elemento absolute é baseado no primeiro componente antecessor **que não seja static**. Logo, caso o elemento absolute esteja dentro de uma `<div>` com `position relative`, é a partir desta `<div>` que o deslocamento será calculado.

```
#square2 {  
  position: absolute;  
  top: 20px;  
  left: 40px;  
}
```





No exemplo acima, a <div> com id square2 foi posicionada com base no corpo da página (<body>), mas deslocada 20 pixels a partir do topo (top) e 40 pixels a partir do lado esquerdo (left).

Note que o elemento do exemplo acima está sobrepondo os outros elementos. Isso acontece pois um elemento absolute deixa de ocupar seu espaço normal no documento e passa a flutuar com uma posição mais livre. Seu espaço no documento é liberado, por isso a <div> verde do exemplo passou a estar ao lado da <div> azul.

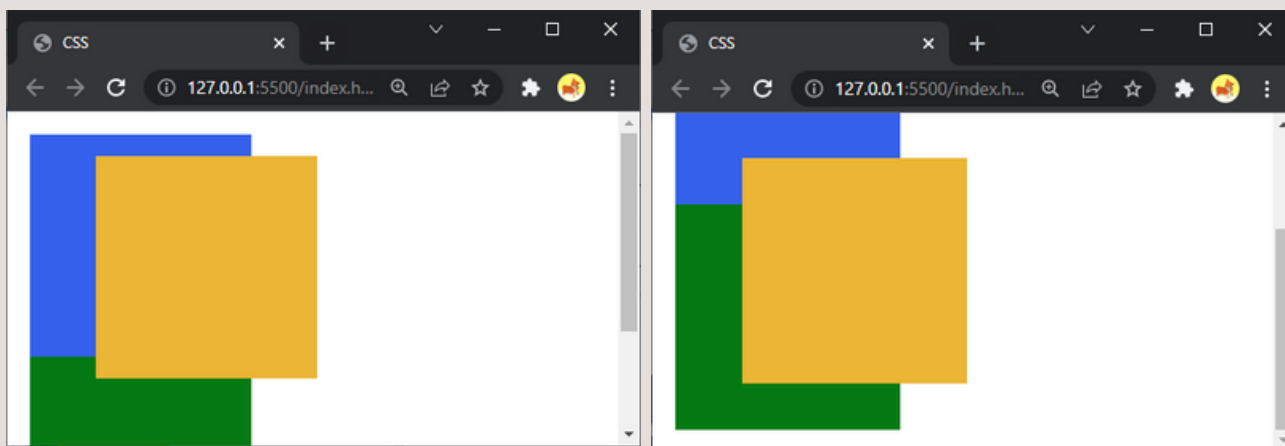
FIXED

A position **fixed** acompanha o mesmo funcionamento da position absolute, de modo que o elemento posiciona-se com base nas propriedades de deslocamento (**top**, **bottom**, **left** e **right**). A diferença entre as duas positions é o elemento base para o posicionamento. Enquanto a position absolute se baseia no primeiro elemento antecessor não static, **a position fixed se baseia na tela em si.**

Ou seja, se definirmos um elemento fixed com deslocamento zerado, o elemento estará no topo da tela independentemente do quanto a página é navegada verticalmente via scroll.

Ao contrário da absolute, que terá seu conteúdo escondido à medida em que a página é movimentada.

```
#square2 {  
  position: fixed;  
  top: 20px;  
  left: 40px;  
}
```



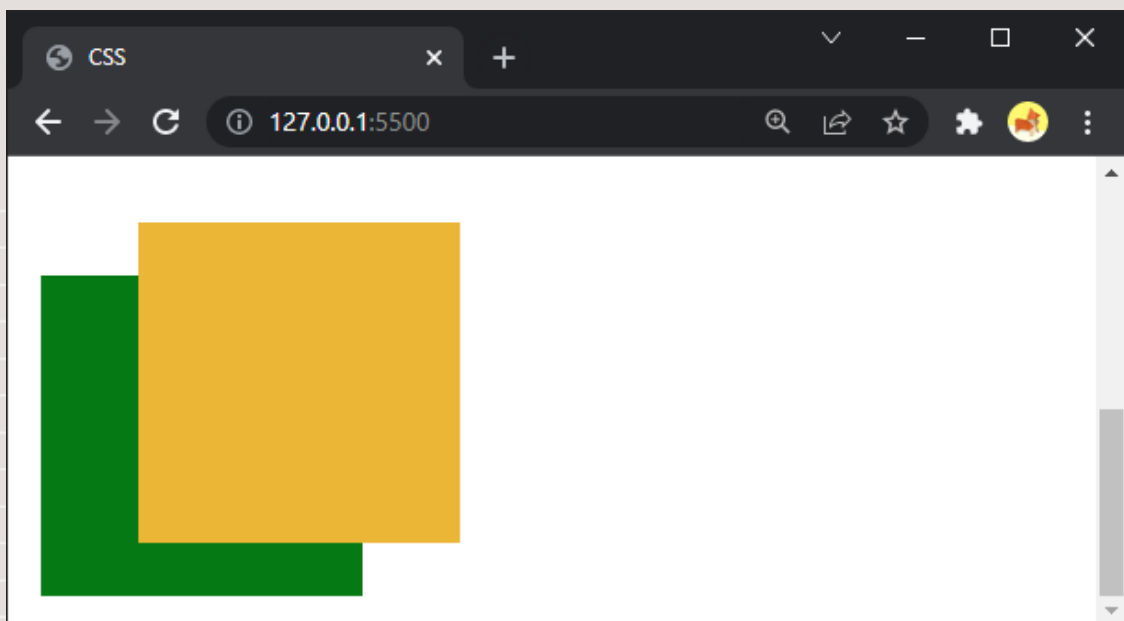
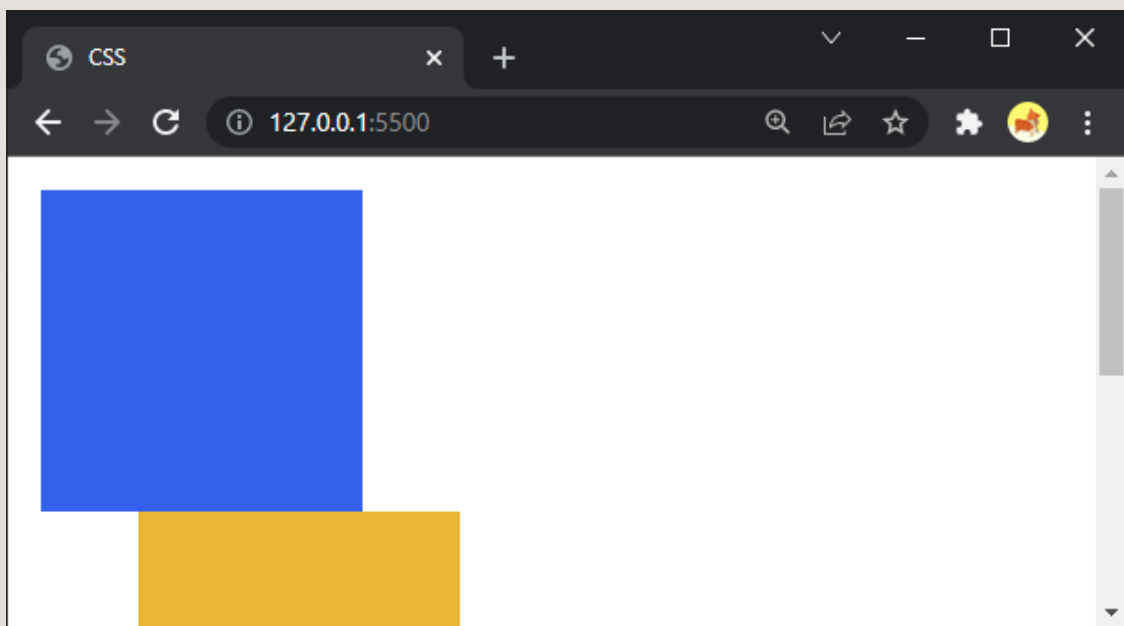
No exemplo acima, o elemento foi fixado a 20 pixels do topo e a 40 pixels do lado esquerdo. Ao rolarmos a página para baixo, o elemento segue na mesma posição em relação a tela: 20px do topo e 40px da esquerda.

STICKY

Unindo as características das `positions relative` e `fixed`, a **sticky** possui um comportamento dinâmico. Enquanto o elemento sticky não estiver visível na tela, sua posição será como um elemento `relative`. Ou seja, irá acompanhar sua posição natural do HTML com a possibilidade de deslocamento.



Assim que a rolagem da página (scroll) atingir o elemento, este apresentará um comportamento fixed. Mas ele só começa a acompanhar o scroll quando a parte superior da tela passa sobre o elemento. A partir de então, se a página seguir a movimentação de scroll, o elemento permanecerá fixo na mesma posição da tela até que um novo fixed seja alcançado.





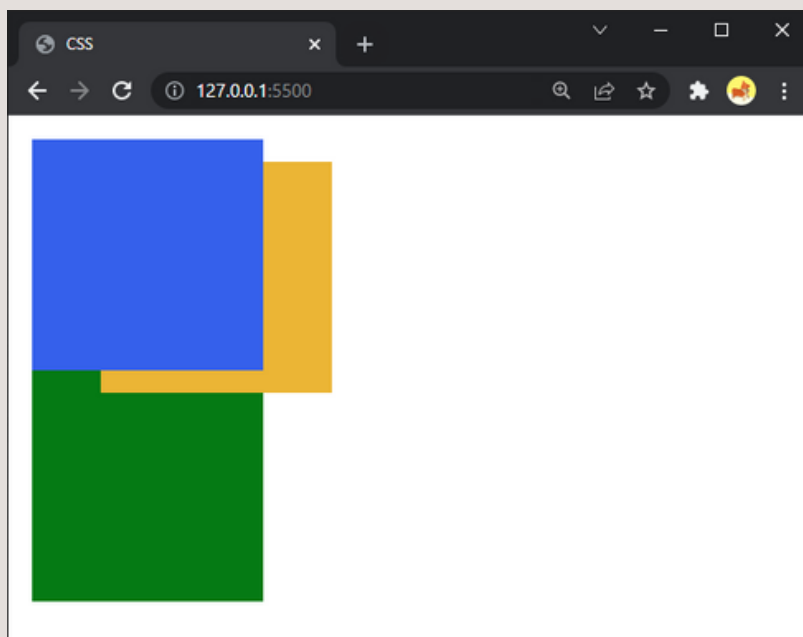
No exemplo acima, a <div> que está no meio possui uma posição relative no início da rolagem da tela. Assim que a rolagem atinge o topo do elemento, ele torna-se fixado e segue acompanhando a rolagem da tela na mesma posição.

PRECEDÊNCIA

Usando as diferentes positions do CSS, eventualmente um elemento poderá ficar encobrindo outros. Com a propriedade **z-index** é possível definir um nível de prioridade para cada elemento. Quanto maior o z-index do elemento, maior é a precedência sobre os outros elementos.

```
#square1 {  
  position: relative;  
  z-index: 2;  
}  
  
#square2 {  
  position: fixed;  
  top: 20px;  
  left: 40px;  
  
  z-index: 1;  
}
```





No exemplo acima, o elemento com id `square1` possui `z-index 2`, enquanto o elemento com id `square2` possui `z-index 1`. Logo, `square1` é exibido com precedência.

OVERFLOW

O **overflow** diz respeito a quando o conteúdo de uma página HTML ultrapassa o tamanho da tela do navegador, seja no eixo X (horizontal) ou no eixo Y (vertical). Além disso, o overflow também é visto quando o conteúdo de um elemento excede o tamanho do seu bloco. Por exemplo, quando uma `<div>` possui `10x10 pixels` e o texto dentro dela é muito grande.

A propriedade `overflow` do CSS permite modificar o comportamento da página e dos elementos nesses casos. Os possíveis valores são: **visible**, **hidden**, **scroll** e **auto**.

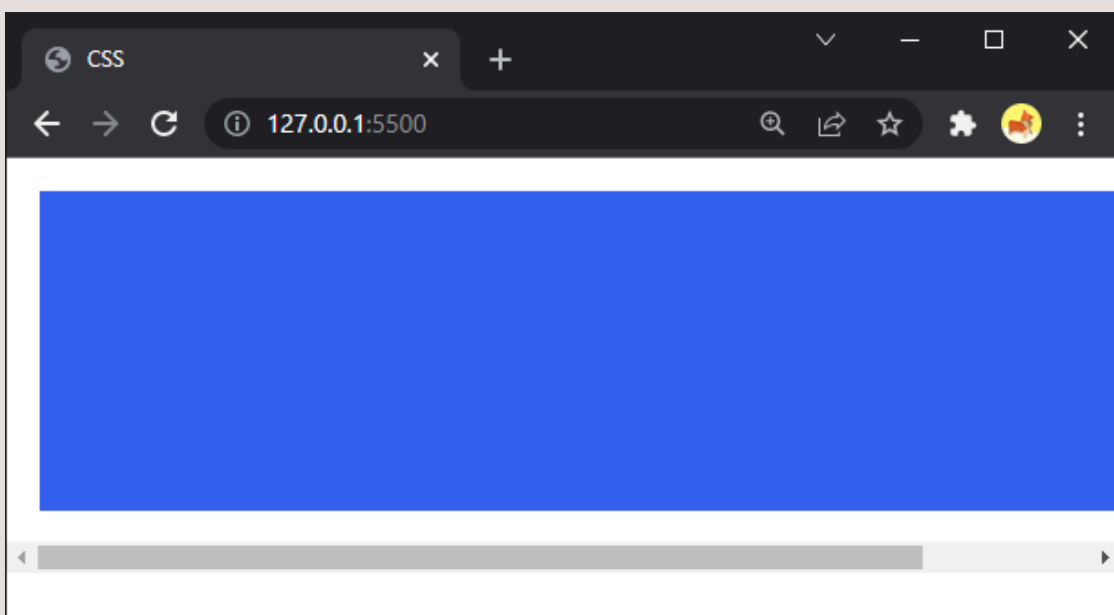
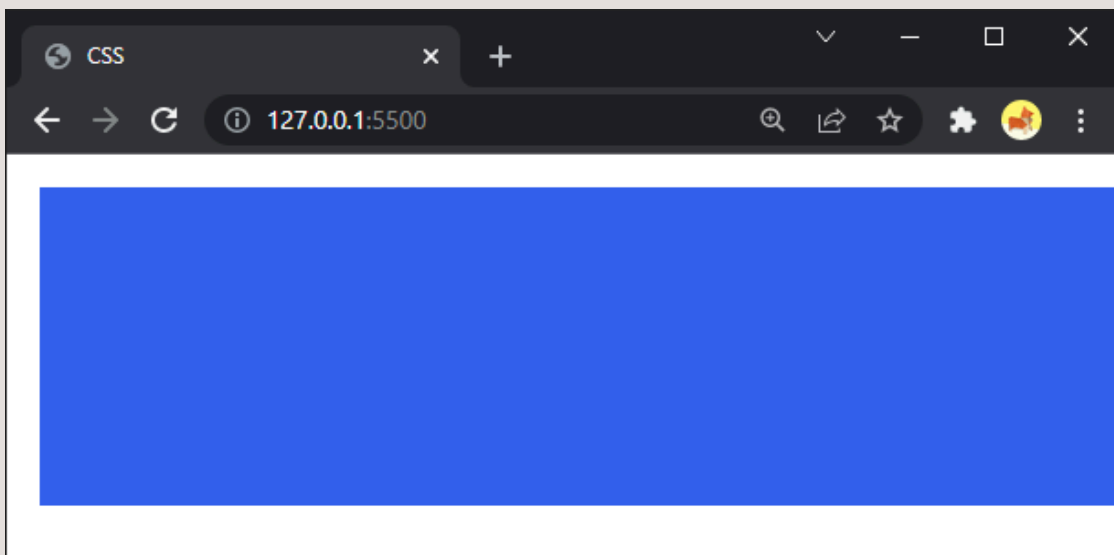


```
#content {  
  overflow: visible;  
  overflow: hidden;  
  overflow: scroll;  
  overflow: auto;  
  
  /* Também podemos definir por eixo */  
  overflow-y: auto;  
  overflow-x: hidden;  
}
```

O comportamento padrão de todos os elementos é o `overflow visible`, em que o conteúdo não deixa de ser exibido caso ultrapasse a fronteira do seu bloco. O valor `hidden` é usado para esconder o conteúdo excedido.

O valor `scroll` esconde o overflow mas adiciona uma barra de rolagem para permitir que o usuário possa visualizar o restante do conteúdo. Mas, neste caso, a barra aparece mesmo sem overflow. O valor `auto` corrige essa situação adicionando o scroll apenas quando há overflow – caso contrário, esconde a barra de rolagem.





No primeiro exemplo acima, o overflow foi definido como hidden, de modo que o bloco foi cortado na medida em que a fronteira com tela do navegador é alcançada. Já no segundo exemplo foi usado o valor auto e a barra de rolagem passou a aparecer, permitindo que o conteúdo seja visualizado na sua integralidade.



FLUTUAÇÃO

No CSS podemos fazer com que elementos sofram flutuação – como se fossem empurrados – para a esquerda ou para a direita. A propriedade usada para esse recurso é o **float**, que recebe os valores **left**, **right** ou **none**.

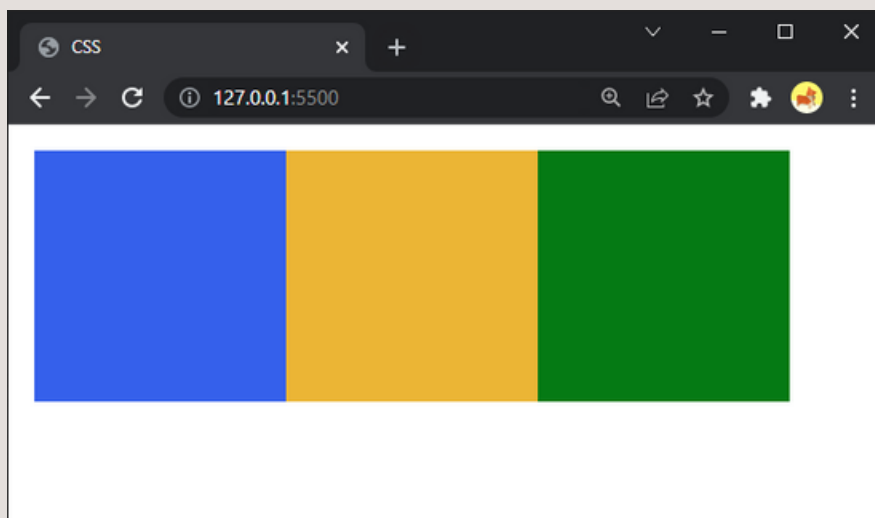
```
div {  
  float: left;  
  float: right;  
  float: none;  
}
```

A flutuação auxilia na organização de elementos sem modificar sua propriedade display. Assim, elementos de bloco podem ser posicionados lado a lado, flutuando-os todos para o mesmo lado. Considere o exemplo abaixo em que três elementos block são empurrados à esquerda.

```
#square1 {  
  float: left;  
}
```

```
#square2 {  
  float: left;
```

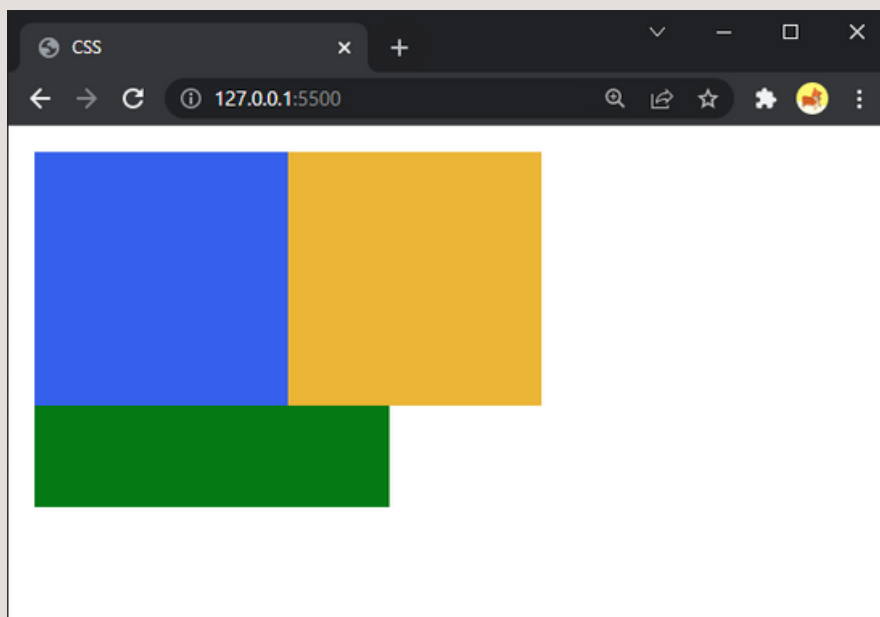
```
#square3 {  
  float: left;  
}
```



É possível perceber que todos os três elementos passaram a ficar lado a lado, como se fossem componentes **inline-block**. Ou seja, quando um elemento é posicionado com float, os demais componentes são posicionados **ao seu redor**.

Mas há uma situação bastante importante quando se usa a propriedade float na página: os elementos flutuantes passam a ser **desconsiderados pelo fluxo normal** dos componentes – como se estivessem de fato flutuando sobre o restante da página. Considere novamente o exemplo abaixo.

```
#square3 {  
  /* float: left; */  
  height: 140px;  
  width: 140px;  
}
```



Neste caso, a `<div>` com id `square3` deixou de ser flutuante pois comentamos a linha com a propriedade `float`. Seu tamanho foi aumentado apenas para uma melhor demonstração. É possível perceber que `square3` fica abaixo das outras `divs`, ignorando suas presenças.

Esse comportamento ocorre pois as outras `divs` estão flutuando sobre a página, de modo que o navegador não os considera no momento de renderizar a `<div>` `square3`.

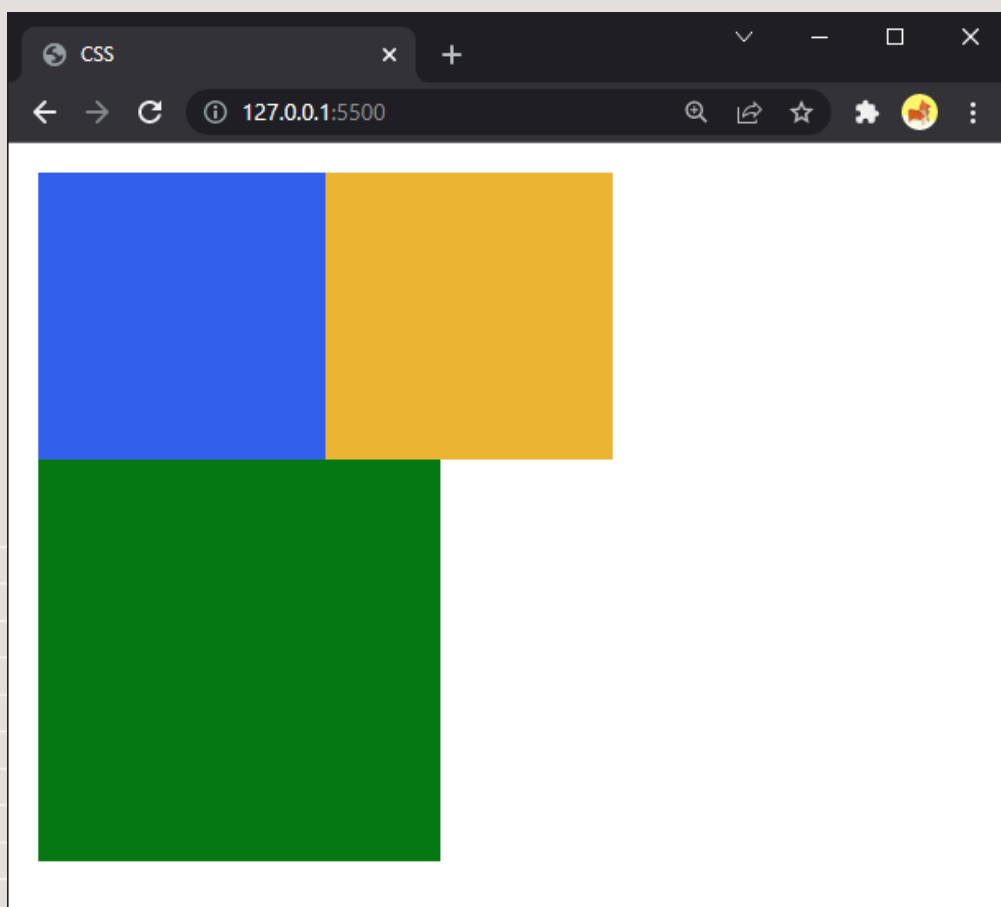
CLEAR

Para resolver o problema abordado acima com os elementos flutuantes que saem do fluxo da página, o CSS oferece a propriedade **clear**. Essa propriedade é usada **no primeiro elemento não float** após algum que seja flutuante, com o objetivo de considerar o espaço dos flutuantes no momento da renderização.



Os possíveis valores para a propriedade clear são: **left**, **right**, **both** e **none**. No caso dos valores left e right, o clear só irá considerar elementos com float left ou right, respectivamente. O valor both considera ambos, seja float left ou right. Já o none é o comportamento padrão e não aplica o clear para qualquer elemento float.

```
#square3 {  
  /* float: left; */  
  height: 140px;  
  width: 140px;  
  
  clear: both;  
}
```





No exemplo acima, ao adicionarmos o clear: both no square3, o navegador passou a considerar os dois elementos flutuantes anteriores e empurrou square3 para baixo, mantendo um fluxo mais natural na página.

REFERÊNCIAS

1. [CSS Position](#)
2. [CSS Z-Index](#)
3. [CSS Overflow](#)
4. [Dicas para trabalhar com a propriedade float do CSS](#)
5. [Propriedade Float e Clear - CSS](#)



BONS ESTUDOS!

